Department of Computer Science Faculty of Information, Communication and Technology, Gujarat Vidyapith			
Sub.:MCA103(Python)	**Student Reg.No.:_____**	**Year : 2025-26**	**Teacher: Prof.(Dr.) Ajay Parikh**

Unit 2: Functions, Modules, Packages, Files, and Regular Expressions

No	Problem Definition	Assignment Date/ Submission Date	Sign of T.A. / Teacher	Grade	Remark
1.	**Scientific Calculator with Functional Extensions** • Build a modular scientific calculator that supports built-in math functions, user-defined functions, and recursion for factorial & power functions. • Implement default, positional, and keyword arguments for advanced operations (e.g., log(base, value) with default base as e). • Package the calculator into a module for reusability.				
2.	**Text Processing Package with Higher-Order Functions** • Develop a package `textutils` with submodules for tokenization, stemming, and frequency analysis. • Use **higher-order functions** (`map`, `filter`, `reduce`) for tasks like stopword removal and frequency aggregation. Integrate **lambda functions** for dynamic filtering (e.g., filter words longer than `n`).				
3.	**Recursive File System Explorer** • Implement a recursive function that lists all files and folders in a given directory. • Add filters using lambda functions + higher-order functions (filter, map) to select files based on size, type, or modified date. • Convert this into a standalone module/package (fsutils).				
4.	**Math Expression Interpreter with Recursion** • Develop an interpreter that evaluates mathematical expressions (like "3*(2+5)") using recursive function parsing. • Integrate lambda-based operators (+, -, *, /) to evaluate dynamically. • Package it as `exprparser` for reusability.				

Department of Computer Science Faculty of Information, Communication and Technology, Gujarat Vidyapith			
Sub.:MCA103(Python)	**Student Reg.No.:_____**	**Year : 2025-26**	**Teacher: Prof.(Dr.) Ajay Parikh**

Additional Programme

5.	**Dynamic Function Loader using Modules** • Write a module loader that imports user-defined functions at runtime. • Allow users to pass function names (strings) and arguments, and execute them dynamically. Handle scope properly (local vs global variables) when importing from external modules.				
6.	**Recursive Data Analyzer (RDA)** • Create a recursive function to traverse nested JSON or dictionary data structures and extract key-value pairs. • Implement both global & local scope tracking to keep record of recursion depth and extracted statistics. • Add lambda-based filters to process specific keys/values.				
7.	**Modular E-Commerce Tax & Discount Engine** • Design a module-based pricing engine for an e-commerce platform. • Use functions with positional, default, and keyword arguments to handle dynamic discounts, GST, and international tax rules. • Package the solution so it can be imported into different projects.				
8.	**Lambda-Driven Data Transformation Pipeline** • Create a data pipeline module where transformations (scaling, normalization, encoding) are passed as lambda functions or higher-order functions. • Demonstrate functional programming concepts by chaining operations dynamically.				

Book Reference: Python Programming, Author: by Rupesh Nasre, AICTE.

Grade	Marks
A	18 - 20
B	14-17
C	11-13
D	8-10
E	5-7
F	1-4
G	Absent